



RAPPORT ARCHITECTURAL : IMPORT CSV BATCH

Réalisé par :
Yahya El Omari

Encadrant :
Abdelmajid Bendrif

23 février 2026

1 INTRODUCTION

Dans le cadre de ce projet, j'ai conçu et développé le module PHP Batch, une solution d'importation de données optimisée pour les environnements en ligne de commande (CLI). L'objectif de ce travail a été de mettre en place un script robuste, capable de traiter de larges volumes de données clients au format CSV et de les intégrer de manière sécurisée dans une base de données MySQL, tout en contrôlant strictement l'empreinte mémoire de l'application.

2 MODÈLE DE DONNÉES ET BASE DE DONNÉES

Le stockage des informations s'effectue au sein de la base de données **cvs_parse**, dans la table **customers**. J'ai opté pour le moteur de stockage **InnoDB** avec l'encodage **utf8mb4**, garantissant ainsi l'intégrité transactionnelle et la compatibilité totale avec les caractères internationaux.

La structure de la table a été pensée pour refléter les données entrantes, comprenant 12 champs principaux :

- **csv_index** : L'index original de la ligne, conservé à des fins de traçabilité.
- **customer_id** : L'identifiant unique attribué au sein du système.
- **first_name** & **last_name** : Les informations nominatives, définies comme obligatoires.
- **company**, **city**, **country** : Les métadonnées géographiques et professionnelles.
- **email** : Le point de contact principal, soumis à une validation stricte.
- Variables additionnelles : **phone_1**, **phone_2**, **subscription_date**, et **website**.

3 ARCHITECTURE GLOBALE

Afin de garantir la maintenabilité et l'évolutivité du code, j'ai structuré l'application en respectant le principe de responsabilité unique (SRP). L'exécution s'articule autour de cinq composants distincts :

- **CsvReader** : Responsable exclusif de l'ouverture et de la lecture itérative du flux CSV.
- **RecordValidator** : Chargé d'appliquer les règles métier pour l'acceptation ou le rejet des entrées.
- **DatabaseFactory** : Gère l'initialisation et la configuration de la connexion PDO.
- **CsvImporter** : Le service central qui orchestre le flux d'extraction, de validation et de chargement.
- **ImportCsvJob** : Le point d'entrée CLI, abstrait par le composant Symfony Console.

4 PROCESSUS MÉTIER DÉTAILLÉ

4.1 1. LECTURE ITÉRATIVE (CSVREADER)

Pour l'extraction des données, j'ai intégré la bibliothèque `League\Csv`. La configuration du `CsvReader` inclut l'appel à `setHeaderOffset(0)`, permettant d'isoler les en-têtes. J'ai privilégié cette bibliothèque car elle retourne un itérateur (`Generator`), évitant ainsi le chargement intégral du fichier en RAM et prévenant les erreurs de type "Out Of Memory" sur de très larges jeux de données.

4.2 2. VALIDATION SANITAIRE (RECORDVALIDATOR)

La protection de l'intégrité de la base de données est assurée par le `RecordValidator`. Chaque ligne extraite est soumise à des vérifications systématiques :

- L'adresse e-mail est imposée et son format est vérifié via le filtre natif `FILTER_VALIDATE_EMAIL`.
- La présence du nom et du prénom est strictement requise.

En cas de non-conformité, plutôt que de lever une exception bloquante, le validateur retourne un rapport d'erreurs. Cette approche non-bloquante permet la continuité de l'importation pour les données valides.

4.3 3. INGESTION PAR LOTS (CSVIMPORTER)

L'optimisation des requêtes d'insertion a constitué une priorité. Dans la classe `CsvImporter`, j'ai mis en place un mécanisme d'insertion par lots (Batching) encadré par des transactions SQL. Le processus se déroule comme suit :

1. Préparation d'une unique requête SQL (`INSERT INTO customers ...`) auprès du moteur de base de données.
2. Accumulation des enregistrements validés dans un tampon en mémoire.
3. Exécution simultanée du lot complet (paramétré par défaut à 100 enregistrements) via la méthode `insertBatch()` et validation de la transaction (`commit()`).

En cas d'erreur de la base de données lors de l'insertion, un `rollback()` est exécuté pour le lot en cours. L'exception est interceptée, l'erreur est historisée, et le processus se poursuit de manière résiliente.

5 COUCHE INTERFACE EN LIGNE DE COMMANDE

5.1 INTÉGRATION SYMFONY CONSOLE

Le déclenchement du script s'effectue via la commande `app:import-csv`, définie dans la classe `ImportCsvJob`. J'y ai injecté les différentes dépendances requises (`CsvReader`, `RecordValidator`) pour instancier dynamiquement le service d'importation central.

5.2 RAPPORT D'EXÉCUTION

La restitution visuelle a été soignée à l'aide de l'utilitaire **SymfonyStyle**. À l'issue du traitement, le terminal affiche de manière structurée les statistiques globales : temps total d'exécution (mesuré via `microtime()`), volume d'enregistrements intégrés, et volume d'enregistrements rejetés. Dans une démarche d'ergonomie, les détails des erreurs rencontrées ne sont affichés à l'écran que si l'utilisateur en confirme explicitement la demande.

6 CONCLUSION

L'architecture implémentée pour ce projet permet d'allier une faible consommation de ressources (grâce à la lecture itérative) à une haute performance d'insertion (via le batching transactionnel PDO). Cette conception dote le module PHP Batch de la robustesse nécessaire pour répondre aux exigences d'un environnement de production appelé à traiter des fichiers de données conséquents.